A Context Package for Search & Recommendations

Team exercise | Agent-Based Software Development | Updated 26 October 2026

What this package contains

1. MODULE.md: our compact session briefing, rendered as one page.

2. Interface review: the real Admin/Marketplace mismatch, our revisions and unresolved decisions.

3. Experiment report: the original A/B runs and conclusion, rendered as two pages.

4. Five-task classification: workflow, lecture pattern or agent, with a reason for each.

5. Five-minute presentation: a short slide deck and timed speaking notes.

Our module is API-only. Storefront owns customer UI/UX. The assignment does not require implementing a complete search engine, webhook system or authentication service.

Current status

The A/B experiment and Part 4 table are complete. Both experimental implementations passed 18 independent checks. The interface review is ongoing: we have real replies and documented mismatches, but shared ownership, IDs and both teams' revised files are not yet confirmed. We must not present that requirement as finished.

The current MODULE.md measures 690 tokens using tiktoken 0.14.0 with o200k_base. The recorded experiment used an older 663-token context, preserved in evidence/frozen-MODULE.md. The later revisions have not been retroactively tested by those runs.

Before submission

Resolve the shared product/article owner and ID, agree the field mapping and confirm both teams update their context files. If unresolved at submission, disclose it and ask how the outstanding review should be completed. Add team-member names if required, confirm the submission channel, and rehearse the five-minute presentation. No external submission has been made.

Source: the professor's two-page Exercise_Module_Context_Package.pdf supplied by the user. Partner-review evidence consists of messages supplied in this chat. Experiment evidence is retained in evidence/ beside this document.

Search & Recommendations

Purpose

We provide APIs for product search, filters, facets, autocomplete and recommendations. Storefront owns customer UI/UX. The exercise uses a small mock-data search task; the full system is not implemented.

Structure

Within src/features/search-recommendations/: schemas/ holds Zod validation; types/ holds data contracts; server/services/ holds search/ranking; server/repositories/ holds data adapters; tests/ holds Vitest tests. These remain scaffolds. server/index.ts is server-only and index.ts is client-safe. Planned thin HTTP routes live under src/app/api/v1/.

Interfaces

Incoming ownership is disputed between Admin and Marketplace; a joint review is open. Do not assume their IDs refer to the same record. Admin proposes { article_id: string, name: string, description?: string, category_id: string, tags: string[], is_published: boolean }. Marketplace proposes { offerId: string, productId: string, sellerId: string, name: string, description: string, category: string, attributes: object, imageUrls: string[], priceMinor: number, currency: string, availability: string, status: string, updatedAt: string }.

For new mocks, use Marketplace's proposal, with EUR prices and string-valued attributes as local assumptions. Its stated availability values are in_stock/out_of_stock; only status=active is public. pending/rejected/blocked/removed are excluded. Marketplace proposes GET /offers?updatedSince=<timestamp>, including blocked/removed changes; final envelope, access, pagination and timestamp rules are pending. One seller per product does not prove one offer per product.

Storefront-facing proposals: GET /api/v1/search with q, category, minPriceMinor, maxPriceMinor, page, pageSize returns { data: { items, total, page, pageSize } }; GET /api/v1/recommendations/similar with productId, limit returns { data: { items } }. Items contain offerId, productId, sellerId, name, category, imageUrls, priceMinor, currency, availability. Result cardinality and Storefront acceptance remain pending. Initial search: trimmed case-insensitive name substring, exact category, inclusive AND price bounds; page=1, pageSize=20, maximum 100. Similarity uses category/attributes, not purchase history.

Conventions

Strict TypeScript; kebab-case files, camelCase functions, PascalCase types. Validate inputs with Zod; keep routes thin and logic testable. Errors use { error: { code, message } }: 400 INVALID_INPUT, 404 PRODUCT_NOT_FOUND for unavailable recommendation source, 503 CATALOG_UNAVAILABLE for source failure. Use Vitest for normal, invalid, boundary and visibility cases. Cross-team access uses HTTP contracts, not internal imports or direct table reads.

Pitfalls

Product/article likely names the same concept; offer adds seller terms, but shared ownership/IDs are not agreed. category is not category_id; attributes are not tags; status is not is_published. Marketplace priceMinor means integer cents including VAT before Marketing discounts. Stock and publication are separate. Co-purchase recommendations need an unassigned purchase-data source. Webhooks, JWT permissions, internal search and reindexing are proposals, not confirmed interfaces. The recorded experiment used an older frozen 663-token context; never rewrite that evidence.

Context measurement: 690 tokens | tiktoken 0.14.0 | o200k_base

Part 2 - Interface review

What we reviewed

We discussed the data Search & Recommendations needs with Marketplace and Admin & Identity. Their replies revealed different assumptions about ownership and field names. A joint group chat was created to resolve them. This is an ongoing real review, not a completed agreement.

What each team said

Marketplace says sellers create their products/offers there, Marketplace generates productId, and Admin moderation decisions are stored with Marketplace. Its proposed offer includes product details, seller ID, price, availability, status and updatedAt. It proposes GET /offers?updatedSince=<timestamp> and will share a draft specification.

Admin proposes an articles table with article_id, name, optional description, category_id, tags and is_published. Admin also said category ownership had not yet been discussed. It then acknowledged uncertainty and created the joint chat. Its later message suggests article and product describe the same item, while an offer adds selling terms. That interpretation has not yet resolved ownership or shared IDs.

Mismatches found

Topic	Different expectations	Our current handling
Ownership and identity	Admin: articles/article_id; Marketplace: products/productId	Keep both proposals documented; require a shared ID or explicit mapping and a named owner.
Categories	Admin: category_id; Marketplace: category label	Do not rename a label into an ID. Agree keys, labels and category ownership.
Search attributes	Admin: tags; Marketplace: attributes	Preserve the distinction; do not assume a lossless mapping.
Visibility	Admin: is_published; Marketplace: status	Marketplace says only active is public. Agree how moderation and publication relate before joining sources.
Price	Teammate proposed float price; Marketplace uses priceMinor	Use integer cents for our proposal; Marketplace says VAT included, before Marketing discounts.
Changes and removal	Marketplace proposes incremental GET; teammate proposes webhooks	Both are proposals; do not claim either is a joint requirement. Agree one integration contract.
Authentication/internal search	Teammate proposed JWT/JWKS, permissions and admin search	No provider agreement is evidenced; keep outside the initial public-search experiment.

Marketplace also states one seller per product, availability in_stock/out_of_stock, and statuses active/pending/rejected/blocked/removed. Changed blocked/removed offers remain in its proposed feed so we can remove them from an index. This is their stated design, not proof of a jointly finalised API.

What we changed

We revised MODULE.md to show both proposals and their unresolved differences. New mock contracts follow the Marketplace proposal as a temporary choice, not as ownership authority. We retained the original experiment fixtures/context unchanged. Storefront remains our intended consumer and owns UI/UX; its acceptance of our result fields has not been received.

What still needs agreement

Agree the product/article owner and shared IDs, the offer relationship, category keys, visibility rules, response envelope, access and update semantics. Confirm whether both teams have updated their interface sections. The assignment requires mismatches to be fixed in both files; we have updated ours, but cannot claim the partner-file step is complete.

Part 3 - Agent experiment

Task and setup

Task: implement simple product-name search with category and price filters, plus tests. The common requirements included case-insensitive matching, inclusive price limits, published products only, deterministic ordering and unchanged input data. No UI, HTTP routes, pagination or recommendations were required.

On 25 October 2026 we used two fresh Codex CLI sessions (0.163.0-alpha.11), the same gpt-6-astra model and medium reasoning setting. Both received identical TypeScript/Vitest starter files and six fictional products. Run A received the task and common setup instructions. Run B additionally received the original MODULE.md, measured at 663 tokens using tiktoken 0.14.0 / o200k_base.

The sessions used separate folders and no resumed conversation. They shared installed dependencies and system/tool context. This was practical separation, not a complete filesystem sandbox between runs. The recorded commands do not show either agent reading the other solution or repository documentation. Both outputs were preserved before evaluation.

Recorded observations

Observation	Run A: task only	Run B: task + context
Assistant messages, including final	3	3
Shell commands / file-change batches	5 / 1	5 / 1
Reported input tokens	110,042	84,425
Cached input tokens, included above	99,968	70,656
Reported output tokens	2,520	2,652
Human corrections during task	0	0
Agent-written tests	14 passed	13 passed
TypeScript check	Passed	Passed
Independent checks	18/18 passed	18/18 passed
Input / output names	query / products	q / items
Filename	searchProducts.ts	product-search.ts
Validation / currency type	Manual RangeError / string	ZodError / EUR literal
Usable as delivered?	Yes, standalone; interface adaptation needed	Yes, standalone; closer to module conventions

Evaluation and findings

The same 18 independent checks covered empty and mixed-case queries, category matching, combined filters, inclusive limits, zero price, invalid bounds, hidden-product exclusion, ordering, summary fields and input immutability. An evaluator adapter mapped A's query/products and B's q/items names; neither implementation was edited to pass the checks.

Both implementations met the common functional requirements. B followed more context-specific choices: q and items, kebab-case filenames, Zod validation and the EUR type. A was not given those conventions, so its alternatives are not functional failures. Neither result is a finished API. Both trust incoming catalog records and need feature integration and HTTP error mapping. B throws ZodError rather than implementing the HTTP error envelope, which was outside the task.

Limitations and later review

We ran before the partner review was resolved, with permission to use a provisional contract. Marketplace and Admin later supplied conflicting ownership and field proposals. The current submission MODULE.md now records that uncertainty; the experiment still refers to the original 663-token context and original fixtures. It does not test the latest schema or resolve Part 2. No results have been relabelled as a new experiment.

This is one A/B pair, not proof of a general effect. The common task and fixtures already provided substantial information. Input-token totals include repeated system/tool context and differ in cache usage, so they do not demonstrate savings caused by MODULE.md. Setup approvals were not coding corrections. A failed sandbox startup occurred before its successful recorded run; B's optional AGENTS.md search returned no matches. Neither required a human code correction.

Conclusion

In this pair, module context improved alignment with our interface and coding conventions while both runs passed all common functional checks. It did not reduce the number of messages or commands. We cannot conclude that context improved general correctness or reduced cost. The practical lesson is to provide a short module briefing, keep it updated after team reviews, and preserve the exact version used for each experiment.

Part 4 - Workflow or agent?

These are recurring tasks in Search & Recommendations. The choice depends on how predictable the steps are. It does not mean the basic search API needs an AI model at runtime.

Recurring task	Best fit	One-sentence reason
Validate incoming catalog data and refresh the index	Workflow	Fetching, validating and updating follow a known sequence that should run deterministically.
Classify a natural-language request as a product lookup, comparison or recommendation	Routing	Classifying the intent selects the specialised handler for that request.
Generate and check candidate search synonyms	Prompt chaining	Extracting terms, proposing synonyms and checking relevance are dependent stages with inspectable outputs.
Improve a ranking configuration against labelled examples	Evaluator and optimizer	Explicit relevance checks guide bounded revisions to the candidate configuration.
Investigate why an expected product is missing from search	Agent	The investigation must choose its next action based on findings about source data, visibility, filters or indexing.

We selected the pattern that fits each task; the exercise does not require using every lecture pattern. Lecture-specific terminology should be checked against the course material when available.

Five-minute presentation

1. Our module - 0:00 to 0:45

We are the Search & Recommendations team. We provide APIs; Storefront builds the customer interface. Our full scope includes search, filters, facets, autocomplete and recommendations. For this exercise, we chose one small task: searching fictional products by name, category and price. We did not build the entire system.

2. Why the interface review matters - 0:45 to 1:45

The review exposed a real mismatch. Admin proposed articles with article_id, category_id, tags and is_published. Marketplace proposed products and offers with productId, category, attributes and status. Both had assumptions about the product data. A joint chat was created to settle ownership and shared IDs. Product and article appear to describe the same concept; an offer adds seller terms, price and availability. That is a working interpretation, not yet a complete agreement. Our revised context records both proposals. We have updated our document, but still need agreement and confirmation that the partner documents are updated too.

3. Our experiment - 1:45 to 2:30

We ran two fresh Codex sessions with the same model, reasoning settings, task and six fictional products. Run A had the task only. Run B also had the original 663-token module briefing. The runs happened before the latest team replies. We kept their original code and context, then ran the same 18 independent checks. The current revised module is not the one used in that historical experiment.

4. What changed - 2:30 to 4:00

Both runs passed all 18 independent checks and TypeScript checking. A wrote 14 passing tests and B wrote 13; those counts do not establish which test suite is better. Both used three assistant messages, five commands and one file-change batch, with no human corrections. The clearest difference was interface alignment. A used query and products; B used the module's q and items. B also used kebab-case files, Zod and an explicit EUR type. A had not seen those conventions, so we do not call its choices functional failures. Both outputs are standalone functions, not complete HTTP APIs.

5. Conclusion - 4:00 to 5:00

For this pair, context helped the agent follow our conventions, but it did not change the common functional pass rate or reduce messages and commands. One pair cannot prove a general improvement or cost saving. Our review also showed that an agent can follow a provisional context correctly even when teams have not agreed the underlying contract. The next step is to close that agreement and keep the context current. For recurring tasks, we chose workflows for index refresh, routing for intent classification, prompt chaining for synonyms, evaluator-and-optimizer for ranking refinement, and an agent for open-ended debugging.